



Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



Classical Cryptography

Cryptography: What



- Plaintext
 - Readable message or data that needs to be protected
- Encryption Algorithm (or Cipher)
 - Algorithm to perform various substitutions and transformations on the plaintext
- Secret key
 - Used as input to the algorithm, transformations depend on the key
- Ciphertext
 - Scrambled message produced as output

Cryptography: Why

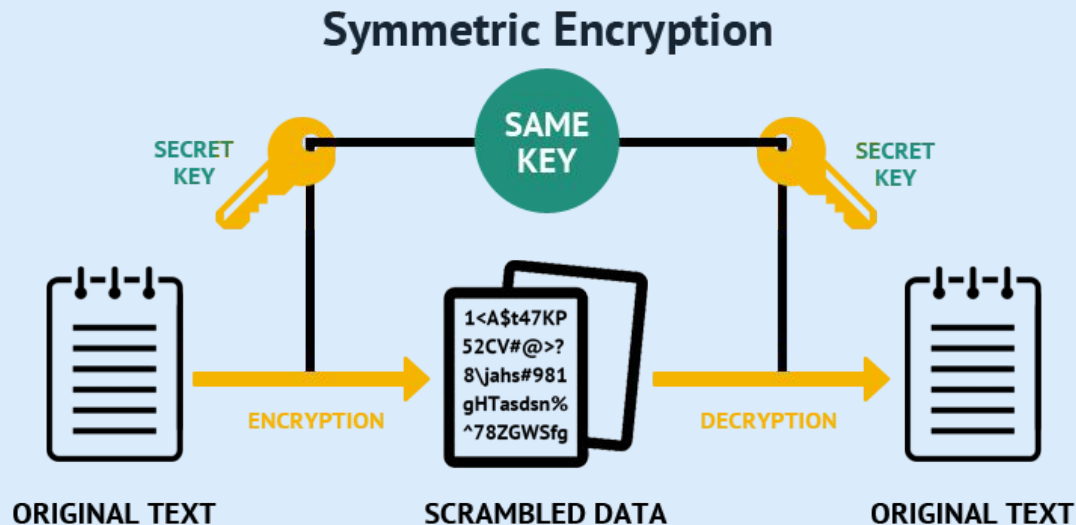


- To achieve several of the important goals in information security:
 - Confidentiality
 - Integrity
 - Authenticity
 - Authorization
 - Non repudiation

Symmetric Crypto



- AKA conventional cryptography
- Sender and receiver must both know **the same secret key**. If the key is somehow revealed to attacker, the protection is lost.



Symmetric Crypto: How?



- There are two primary operations. Note that these should be perfectly reversible!

(1) Substitution



Replace character(s) or bit-strings with other characters

(2) Permutation



Change the order of characters or bit strings.

Symmetric Crypto Limitations



Pros

- Very fast to compute. Hardware acceleration available in many cases

Cons

- No mechanism of sharing the key secretly. We need an alternate **secure channel** for key sharing.
- Managing separate keys for each **pair** of users, otherwise impersonation is possible
 - If Alice and Bob share a key. Imagine Trudy shares the same key with Alice for secure communication. Trudy may act as Alice and talk to Bob.

Classical Ciphers



- We will start with studying classical ciphers – those used before the era of computers.
- Computers can break these ciphers easily, that's why they are no longer used.

Substitution ciphers

Replace plaintext characters
according to a translation table.



Monoalphabetic substitution



CIPHER ALPHABET

A = B	H = A	O = O	V = L
B = V	I = D	P = Y	W = P
C = G	J = Z	Q = F	X = U
D = Q	K = C	R = J	Y = I
E = K	L = W	S = X	Z = R
F = M	M = S	T = H	
G = N	N = E	U = T	

Figure 1

Classical Substitution Ciphers



Question

What is the size of 'key space' in the monoalphabetic substitution cipher assuming 26 letters?

- a. 26
- b. 26!
- c. 2^{26}
- d. 26^2

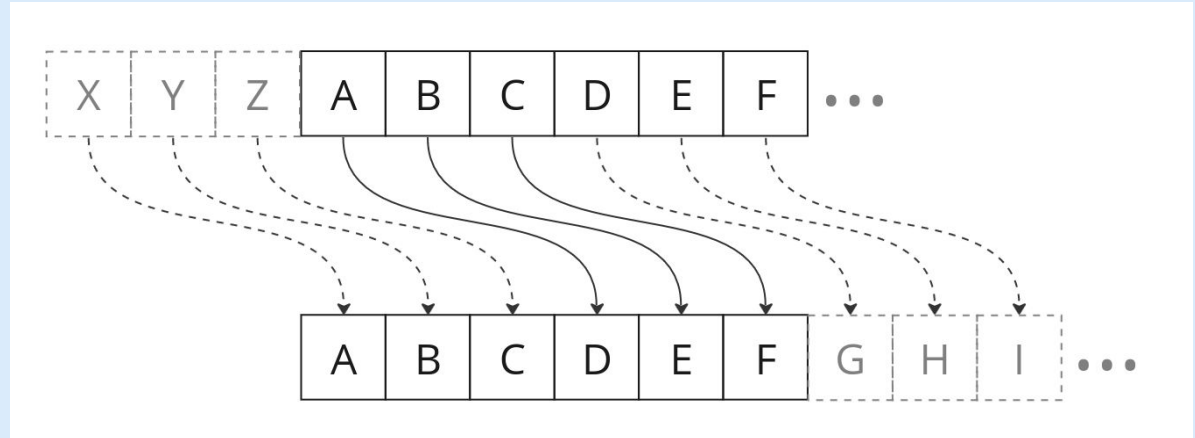
Key space = set of all possible keys

Classical Substitution Ciphers



- Most well-known example: **Caesar Cipher**
- Shifts each letter in the plaintext a fixed number of positions down/up the alphabet.
- Number of shifts becomes the secret key.

Example with key=3



**How many possible
keys are there?**

When key=13, cipher is
called **ROT-13**

Attacking the Ceaser cipher



- Ceaser cipher can be quite easily broken by a brute-force attack: The attacker tries every possible key on a piece of ciphertext until an intelligible translation into plaintext is obtained.
- On average, half of all possible keys must be tried to achieve success.
 - if there are X different keys, on average an attacker would discover the actual key after $X/2$ tries.

Attacking the Ceaser cipher



ciphertext

phhw ph diwhu wkh wrjd sduwb

KEY

1	oggv	og	chvgt	vjg	vqic	rctva
2	nffu	nf	bgufs	uif	uphb	qbsuz
3	meet	me	after	the	toga	party
4	ldds	ld	zesdq	sgd	snfz	ozqsx
5	kccr	kc	ydrpc	rfc	rmey	nyprw
6	jbbq	jb	xcqbo	qeb	qldx	mxoqv
7	iaap	ia	wbpan	pda	pkcw	lwnpu
8	hzzo	hz	vaozm	ocz	objv	kvmot
9	gyyn	gy	uznyl	nby	niau	julns
10	fxxm	fx	tymxk	max	mhzt	itkmr
11	ewwl	ew	sxlwj	lzw	lgys	hsjlg
12	dvvk	dv	rwkvi	kyv	kfxr	grikp
13	cuuj	cu	qvjuh	jxu	jewq	fqhjo

14	btti	bt	putg	iwt	idvp	epgin
15	assh	as	othsf	hvs	hcuo	dofhm
16	zrrg	zr	nsgre	gur	gbtn	cnegl
17	yqqf	yq	mrfqd	ftq	fasm	bmdfk
18	xppe	xp	lqepc	esp	ezrl	alcej
19	wood	wo	kpdob	dro	dyqk	zkbdi
20	vnnv	vn	jocna	cqn	cxpj	yjach
21	ummb	um	inbmz	bpm	bwoi	xizbg
22	tlla	tl	hmaly	aol	avnh	whyaf
23	skkz	sk	glzlx	znk	zumg	vgxze
24	rjjy	rj	fkyjw	ymj	ytlf	ufwyd
25	qiix	qi	ejxiv	xli	xske	tevxc

Brute forcing in general



- Preceding example shows that there is more to brute-force than simply running through all possible keys.
- The analyst must be able to recognize plaintext as plaintext.
 - If the message is just plain text in English (or another language), then the result is easily understood by a human reader, but the task of recognizing the language would have to be automated.
 - If the text message has been compressed before encryption, then recognition is more difficult.
 - And if the message is some more general type of data, such as a word document or image or audio, the problem becomes even more difficult to automate.

One-Time Pad (OTP)



- To implement OTP, for each message use a **random key** that is at least as long as the message itself.
- Combine the key with the plaintext (e.g. similar to Vigenère cipher) to generate ciphertext.
- Each key is to be used to encrypt and decrypt a single message, and then is **discarded** .
 - Each new message requires a new key of the same length as the message.

One-Time Pad (OTP)



The OTP name comes from the paper-pads on which random key streams were printed.

Two copies of each pad had to be created, one for sender other for receiver.

Each paper sheet was destroyed after use.



<https://www.cryptomuseum.com/crypto/otp/index.htm>

OTP Strength



When **applied correctly** , the OTP provides a **truly unbreakable** cipher.

Why?

- It produces random output that bears no statistical relationship to the plaintext.
- Ciphertext contains no information whatsoever about the plaintext

OTP Strength: Example



ciphertext **rkrbb ntiin dfsnw ahlgz dit**

guess the key

mkgqt anqpn mnhfd gsszv lyv

falling stars lit up the sky

a plausible plaintext

another
key guess

mcaxb upacg kxfup wvxpm vvn

fire at eight in the morning

another plausible plaintext

This way, a given ciphertext maps to a very large number of possible plaintexts, and attacker will have no clue which key is the correct one.

OTP Practical or not?



OTP implementation rules are very strict. If any one of these rules is violated, we no longer have the guarantee of unbreakability.

1. Key should consist of truly random characters (not pseudo-random).
2. Key should be as long the plaintext (or longer).
3. Only two copies of the key should exist.
4. A key should be used only once.
5. Both copies of the key are destroyed immediately after use.

In practice it is extremely challenging to

- generate large amounts of truly random keys (millions of characters); and...
- securely distribute those random keys

OTP Practical or not?



To illustrate the impracticality of the OTP, consider a simple example. Suppose Alice wants to send a 100 MB file to Bob using the OTP. She must first generate a 100 MB random key, which she then uses to encrypt the 100 MB file to produce the ciphertext. This 100 MB key must be securely transmitted to Bob before he can decrypt the ciphertext. If Alice and Bob wish to communicate regularly, they would need a new 100 MB key for each message, resulting in an enormous amount of key data that must be securely generated, stored, and transmitted.